

meeting 2026-09-29

2026-09-29

Outline

1. The full GuideBender model
2. How we get our computational datasets
3. Computational scaling results

1. The full GuideBender model

In this section I give a full end-to-end description of the GuideBender model, as it is actually implemented in the code, rather than the simplified version presented in the paper.

Let $g \in \{1, 2, \dots, G\}$ index guide and $n \in \{1, 2, \dots, N\}$ index cell. N_{gn} denotes the observed UMI count of guide g in cell n , i.e. the actual entry of the gRNA matrix. We model it as coming from three separate sources:

$$N_{gn} = N_{gc}^{\text{signal}} + N_{gn}^{\text{endo}} + N_{gn}^{\text{exo}}.$$

These respectively represent the contribution of actual expression; ambient noise; and sequencing errors like barcode swaps or PCR chimeras. Cell n 's total number of infections K_n is given by a HurdlePoisson, and this total number is distributed among the available guides (multiple infections are possible).

To actual sample from this model, we can group the parameters and quantities as follows:

1. Guide-specific:

These quantities are drawn once per dataset, and shared between cells.

key quantities:

- $\mathbf{w} \in [0, 1]^G$ gives a probability distribution over guides, representing relative infection frequency.
- $\mathbf{d}^{\text{guide}} \in (0, \infty)^G$ controls the magnitude of each guide's signal UMI counts
- $\chi^a \in [0, 1]^G$ gives a different probability distribution over guides, representing the relative appearance of each guide in the ambient soup. It is based on both $\mathbf{w}$ and $\mathbf{d}^{\text{guide}}$ (since ambient quantity depends on infection frequency and expression amount).

These are constructed as follows:

$$\mathbf{w} \sim \text{Dirichlet}(\alpha \mathbf{1}_G)$$

so the parameter α governs how uniform $\mathbf{w}$ is.

$$\log d_g^{\text{guide}} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_{\text{guide}}^2)$$

so σ_{guide}^2 determines the spread of the guide magnitudes.

Finally, for χ^a , we begin by constructing the distribution $\mathbf{p}$ over guides, which combines $\mathbf{w}$ and $\mathbf{d}^{\text{guide}}$:

$$\mathbf{p} := \frac{\mathbf{w} \odot \mathbf{d}^{\text{guide}}}{\mathbf{w}^T \mathbf{d}^{\text{guide}}}.$$

Next, we introduce parameters $f \in [0, 1]$ and $s > 0$, called the endogenous shape parameters, and we construct

$$\mathbf{a} := (1 - f) G s \mathbf{p} + f s \mathbf{1}_G$$

so $\mathbf{a}$ is a convex combination of $G\mathbf{p}$ and $\mathbf{1}_G$ (both sum to G), with f pulling us towards symmetry, then scaled by s to further control concentration. Finally, we obtain

$$\chi^a \mid \mathbf{w}, \mathbf{d}^{\text{guide}} \sim \text{Dirichlet}(\mathbf{a}).$$

In practice, in both the `fishash` code and my code, we always use $\sigma_{\text{guide}}^2 = \frac{1}{4}$, $s = 1$, and $f = 0$. So really we have

$$\chi^a \mid \mathbf{w}, \mathbf{d}^{\text{guide}} \sim \text{Dirichlet}\left(G \frac{\mathbf{w} \odot \mathbf{d}^{\text{guide}}}{\mathbf{w}^T \mathbf{d}^{\text{guide}}}\right),$$

so the relative amounts of each guide in the ambient soup directly depend on their relative frequency and expression magnitudes (in expectation; the realized χ^a can vary from this).

In my computational scaling datasets, α is the only one of these parameters that is adjusted to fit a real dataset.

2. Cell-specific

- $K_n \in \mathbb{N}$ is the total number of infections in cell n . We use $K_n \stackrel{\text{iid}}{\sim} \text{HurdlePois}(\pi, \lambda)$, so $P(K_n = 0) = \pi$, and K_n is from a zero-truncated Poisson with rate λ otherwise.
- From K_n we get $\mathbf{J}_n$, the actual number of each infection: $\mathbf{J}_n \mid K_n, \mathbf{w} \sim \text{Mult}(K_n, \mathbf{w})$ [this is the guide-level parameter $\mathbf{w}$, so technically this is not purely cell-level, but it makes sense here]
- $\mathbf{d}^{\text{cell}} \in (0, \infty)^N$: cell-level scaling factors for signal UMIs.
- $\mathbf{d}^{\text{drop}} \in (0, \infty)^N$: cell-level scaling factors for ambient (endo) UMIs.
- $\rho_n \in [0, 1]$ determines what fraction of a cell's UMIs come from exogenous error vs droplet measurements. $\rho_n = 0$ means that the observed UMI count is entirely droplet, while $\rho_n = 1$ means the observed UMI count is entirely exogenous error.
- $\epsilon_n > 0$ determines the capture efficiency of each droplet, and scales the mean of every term. We have $\epsilon_n \stackrel{\text{iid}}{\sim} \text{Gamma}(\kappa, \kappa)$ (shape-rate) where $\kappa = 50$ is always used in practice, producing a bell-shaped curve centered at 1.

Additional details:

We need some user-provided inputs to actually obtain the parameters for these quantities. We need

- `snr`, the signal:noise ratio, which controls how much of the total per-cell UMI budget goes to signal
- `frac_noise_endo`, the fraction of the error that is endogenous
- `count_per_cell`: the target median (within each 1000 cell chunk, to be discussed later) of each cell's total UMI over all sources (signal, endo, and exo)
- π and λ
- `rho_sum` (controlling ρ_n 's concentration; technically user-provided, but always set to 10 in practice)

From these, we derive the following:

$$\phi_s = \frac{\text{snr}}{1 + \text{snr}}, \quad \phi_e = \text{frac_noise_endo} \cdot (1 - \phi_s), \quad \phi_x = 1 - \phi_s - \phi_e$$

These give a partition of total per-cell UMIs into signal, endo, and exo.

For the scaling factors $\mathbf{d}^{\text{cell}} \in (0, \infty)^N$ and $\mathbf{d}^{\text{drop}} \in (0, \infty)^N$, we have

$$\begin{aligned} \log d_n^{\text{cell}} &\stackrel{\text{iid}}{\sim} \mathcal{N}\left(\mu_{\text{cell}}, \frac{1}{2^2}\right) \\ \log d_n^{\text{drop}} &\stackrel{\text{iid}}{\sim} \mathcal{N}\left(\mu_{\text{drop}}, \frac{1}{2^2}\right). \end{aligned}$$

The parameters of μ_{cell} and μ_{drop} are then derived in a pretty complicated way. We do it by using the fact that the ratio of signal to endo should be around ϕ_s/ϕ_e , while the per-cell UMI total should be around `count_per_cell`. Hard-coding all lognormal variance terms to $1/4$, we get

$$\begin{aligned} \mu_{\text{drop}} &= \log\left(\frac{C \phi_e}{\phi_s + \phi_e}\right) - \frac{1}{8}, \\ \mu_{\text{cell}} &= \log\left(\frac{C \phi_s}{(\phi_s + \phi_e) \mu_K}\right) - \frac{1}{4} \end{aligned}$$

where C is `count_per_cell` and μ_K , the expected number of infections, can be shown to be

$$\mu_K = \frac{(1 - \pi)\lambda}{1 - e^{-\lambda}}.$$

The GuideBender code seems to have a bug where one of the lognormal variances is missing the $\div 2$, although the chunk median rescaling prevents it from mattering.

Then for ρ_n , we compute

$$\rho_\alpha = \phi_x \cdot \text{rho_sum}, \quad \rho_\beta = \text{rho_sum} - \rho_\alpha$$

and realize

$$\rho_n \stackrel{\text{iid}}{\sim} \text{Beta}(\rho_\alpha, \rho_\beta), \quad \mathbb{E}[\rho_n] = \frac{\rho_\alpha}{\rho_\alpha + \rho_\beta} = \phi_x.$$

3. cell and guide level

Everything in this section is an $G \times N$ matrix indexed by guide g and cell n . For now we describe it as if all N cells were simulated together; chunking is added at the end.

The expected signal for guide g in cell n is

$$\mu_{gn}^s = J_{gn} d_g^{\text{guide}} (1 - \rho_n) \epsilon_n d_n^{\text{cell}}.$$

This is the only place d^{guide} appears.

They use a Poisson-Gamma formulation to get the NegBin signal distribution. Draw the signal rate λ_{gn}^s as

$$\lambda_{gn}^s \mid \mu_{gn}^s \sim \text{Gamma}\left(\text{shape } \frac{1}{\Phi_{\text{cell}}}, \text{scale } \Phi_{\text{cell}} \mu_{gn}^s\right),$$

so that $\mathbb{E}[\lambda_{gn}^s \mid \mu_{gn}^s] = \mu_{gn}^s$ and $\text{Var}(\lambda_{gn}^s \mid \mu_{gn}^s) = \Phi_{\text{cell}} (\mu_{gn}^s)^2$. We always use $\Phi_{\text{cell}} = 1$, which makes λ_{gn}^s exponential with mean μ_{gn}^s . When $J_{gn} = 0$ the scale is 0 and $\lambda_{gn}^s = 0$, so uninfected guides get no signal.

For endo, the expected endogenous (ambient) noise is

$$\lambda_{gn}^e = \chi_g^a (1 - \rho_n) \epsilon_n d_n^{\text{drop}}.$$

This is the only place χ^a appears. This term also does not depend on the infection status of cell n .

For exo, first add the two “droplet” (endogenous) sources:

$$\lambda_{gn}^{\text{endo}} = \lambda_{gn}^s + \lambda_{gn}^e,$$

and take its row totals, column totals and grand total:

$$R_g = \sum_n \lambda_{gn}^{\text{endo}}, \quad C_n = \sum_g \lambda_{gn}^{\text{endo}}, \quad S = \sum_g R_g.$$

In the code, they compute the expected exogenous noise by

$$\lambda_{gn}^x = \frac{R_g C_n}{S} \cdot \frac{\rho_n}{1 - \rho_n}.$$

It’s definitely what the code does, but I haven’t verified correctness. Claude thinks it’s good though.

Taking that as correct, we have the total noise rate as

$$\lambda_{gn}^n = \lambda_{gn}^e + \lambda_{gn}^x.$$

Next, we will enforce that `count_per_cell` really is the median. We do this by effectively dividing the means by their median and then multiplying by `count_per_cell`:

$$r := \frac{\text{count_per_cell}}{\text{median}_n \sum_g (\lambda_{gn}^s + \lambda_{gn}^n)}, \quad \text{set } \lambda_{gn}^s \leftarrow r \lambda_{gn}^s, \quad \lambda_{gn}^n \leftarrow r \lambda_{gn}^n.$$

After this, the median over cells of the total **expected** count equals `count_per_cell` exactly; the median of the realized counts will be close but not equal. Because the same r multiplies everything, it sets the overall level without changing any proportions (signal vs noise, guide vs guide, cell vs cell). This is why only the difference $\mu_{\text{cell}} - \mu_{\text{drop}}$ from Section 2 matters in practice.

Finally, to realize this model, we sample

$$N_{gn}^{\text{signal}} \sim \text{Pois}(\lambda_{gn}^s), \quad (N_{gn}^{\text{endo}} + N_{gn}^{\text{exo}}) \sim \text{Pois}(\lambda_{gn}^n),$$

since λ_{gn}^n is the mean of the endo and exo together. These are drawn independently over all g, n given the rates. The marginalization gives us the NegBin for the signal.

Note that an infection can still have $N_{gn} = 0$. There might always be a gap between the true MOI and the MOI as we would ever estimate it in practice (guide-cell pairs with 0 UMIs are never getting called perturbed).

In practice the N cells are split into chunks of 1000 (`chunk_cells`), and everything in this section is computed separately in each chunk. The guide-level quantities ($\mathbf{w}$, d^{guide} , χ^a) and cell-level quantities (K_n , $\mathbf{J}_n$, ρ_n , ϵ_n , d_n^{cell} , d_n^{drop}) are drawn once for all cells beforehand. Chunking changes exactly two things:

1. In the exogenous term, R_g , C_n and S are totals within the chunk, not over all N cells. C_n is a per-cell total, so it is unaffected; R_g/S becomes the chunk’s own endogenous guide profile.
2. The median in r is over the chunk’s 1000 cells, so each chunk gets its own r , and `count_per_cell` is hit chunk by chunk.

2. Computational scaling datasets

We want to use a real dataset to pick good GuideBender parameters, while satisfying the realism requirements that I previously presented. These are

1. MOI stays constant
2. per-guide true perturbations doesn't vary with N and G
3. the per-cell total signal and error UMI counts do not meaningfully vary with N and G

As a consequence, we use a sequence of datasets with N and G fixed.

The following GuideBender parameters are never changed:

Code argument	Symbol	Value	Role
Phi_cell	Φ_{cell}	1	Gamma overdispersion of the signal rate λ_{gn}^s
Phi_noise	Φ_{noise}	0	Gamma overdispersion of the noise rate λ_{gn}^n (0 = none)
rho_sum	$\rho_\alpha + \rho_\beta$	10	Concentration of $\rho_n \sim \text{Beta}(\rho_\alpha, \rho_\beta)$
eps_alpha	κ	50	Shape and rate of $\epsilon_n \sim \text{Gamma}(\kappa, \kappa)$
d_sigma_guide	σ_{guide}	0.5	SD of $\log d_g^{\text{guide}}$
d_sigma_cell	σ_{cell}	0.5	SD of $\log d_n^{\text{cell}}$
d_sigma_drop	σ_{drop}	0.5	SD of $\log d_n^{\text{drop}}$
endo_shape_sum	s	1	Total concentration of the Dirichlet for χ^a
endo_shape_flat	f	0	Uniform component of the Dirichlet for χ^a
use_median	—	TRUE	Rescale rates so the chunk median of per-cell totals is <code>count_per_cell</code>

The following are the parameters we set:

Code argument	Symbol	How we set it	Role
n_guides	G	Per rung: $\text{round}(N \cdot G_0 / N_0)$	Number of guides
n_cells	N	Per rung: chosen	Number of cells
moi	λ	Calibrated per rung	Rate of the Poisson in the hurdle Poisson for K_n
hurdle_prob	π	Measured from the real matrix	$P(K_n = 0)$
snr	—	Calibrated once per regime	Sets $\phi_s = \text{snr} / (1 + \text{snr})$
count_per_cell	C	Measured from the real matrix	Target median of per-cell totals
guide_infection_alpha	α	Measured from the real matrix	Concentration of $\mathbf{w} \sim \text{Dirichlet}(\alpha \mathbf{1}_G)$
frac_noise_endo	—	Chosen: 0.75	Endogenous share of the noise; sets ϕ_e and ϕ_x . Very important but hard to pick.
chunk_cells	—	Chosen: 1000	Cells per chunk, avoids a $G \times N$ dense matrix

How we set or chose these:

Let Y be the real count matrix, with G_0 guides and N_0 cells. We need to know which guide-cell pairs are perturbed, so we use thresholding. Gasperini gets a threshold of 5, and Replogle of 10.

Measured from the real matrix

- **count_per_cell** (C): the median of the per-cell UMI totals
- **hurdle_prob** (π): a cell with no guides flagged as perturbed in our assignment is determined to have $K_n = 0$, so we estimate π as the fraction of cells with zero UMIs hitting the threshold.
- **guide_infection_alpha** (α): [Claude did this] let m_g be the number of cells in which guide g is called perturbed, with mean $\bar{m}$ and coefficient of variation CV_m across the G_0 guides. Subtract the sampling variance of the counts, $CV_w^2 = CV_m^2 - 1/\bar{m}$, and solve the Dirichlet relation $CV_w^2 = (G_0 - 1)/(G_0\alpha + 1)$ for α :

$$\alpha = \frac{1}{G_0} \left(\frac{G_0 - 1}{CV_w^2} - 1 \right).$$

- **n_guides** (G): for each rung's chosen number of cells N , $G = \text{round}(N \cdot G_0/N_0)$, which keeps the real cells-per-guide ratio.

Calibrated by simulation

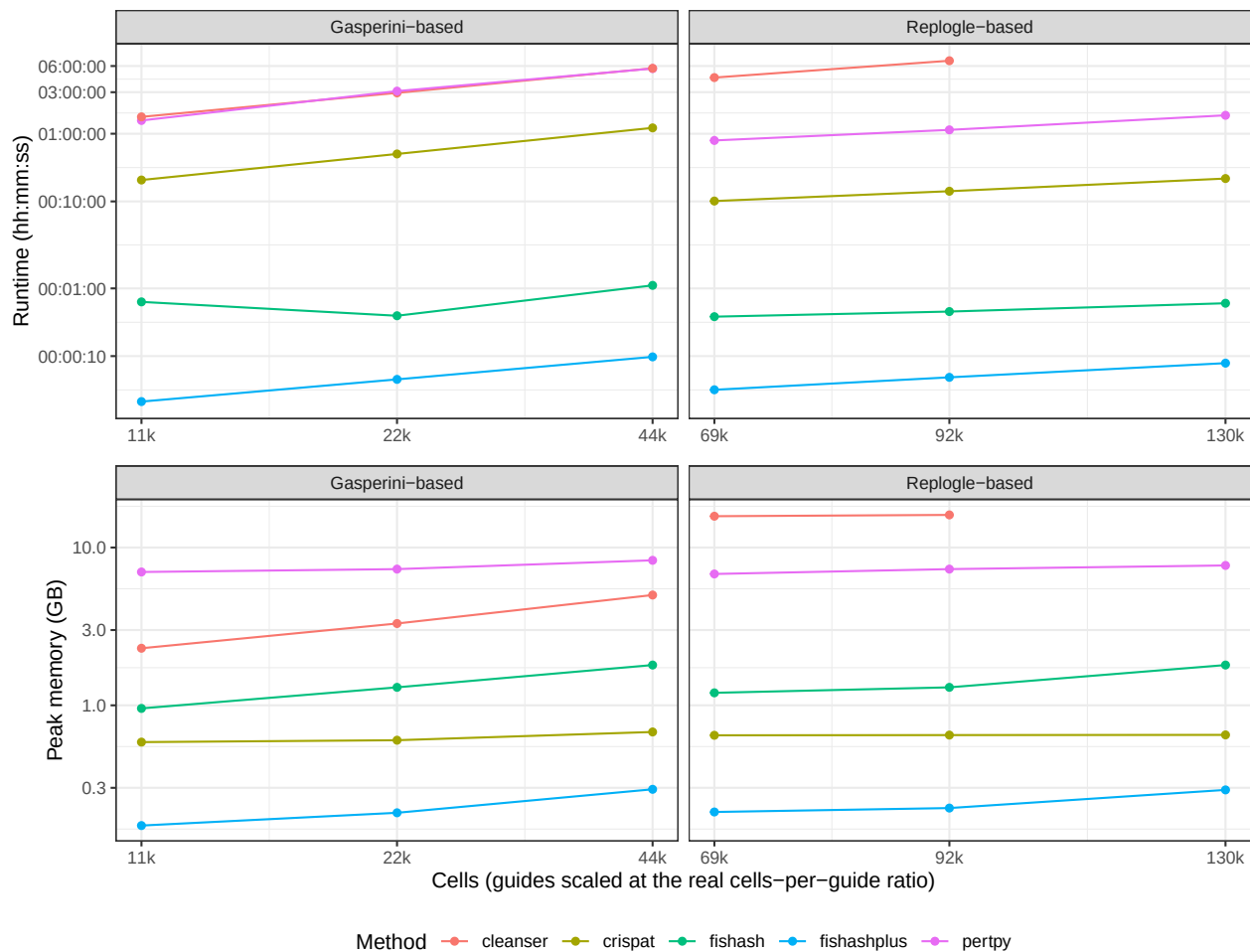
Two statistics are matched, each computed the same way on real and simulated matrices: the thresholded MOI and the number of nonzeros per cell,

$$M_t = \frac{1}{N} \sum_{g,n} \mathbb{1}\{Y_{gn} \geq t\}, \quad Z = \frac{1}{N} \sum_{g,n} \mathbb{1}\{Y_{gn} > 0\}.$$

Each evaluation simulates n_{cal} cells (4,000 for Gasperini, 10,000 for Replogle) and averages M_t and Z over n_{seeds} independent simulations (1 for Gasperini, 4 for Replogle). These use seeds 11, 12, ..., and the same seeds are reused at every evaluation. All other inputs are set as above.

- **snr** (Stage 1): solved jointly with λ at G^* , the largest rung's number of guides.
 - Starting values: **snr** is the real matrix's mean per-cell UMIs in entries called perturbed, divided by its mean per-cell UMIs in nonzero entries not called perturbed; $\lambda = M_t/(1 - \pi)$.
 - Each pass first solves for λ so that the simulated M_t equals the real value, holding **snr** fixed, then solves for **snr** so that the simulated Z equals the real value, holding λ fixed. Each solve is a one-dimensional root-finding (**uniroot**) on the log scale.
 - Passes stop once both M_t and Z are within a relative tolerance of their targets (0.5% for Gasperini, 1% for Replogle); after 6 passes without convergence, the code stops with an error.
 - The resulting **snr** is used at every rung. The λ from this stage is only a starting value for Stage 2.
- **moi** (λ) (Stage 2): with **snr** fixed at its Stage 1 value, λ is solved separately at each rung's G so that the simulated M_t equals the real value. The search starts from the Stage 1 λ at the first rung and from the previous rung's solution after that. The code stops with an error if M_t is not within tolerance. Z is not a target in this stage.

results



Most of this looks as expected. fishash+ does great. The one concerning thing is how CLEANSER jumps way up in memory for the Replogle data, but this turns out to be a genuine result. CLEANSER is very sensitive to the guide with the most non-zeros. The real Replogle has a guide that is about 17% non-zero, which is very dense, and leads to huge memory use on Replogle. These simulated datasets preserve that property, which is why Replogle is so costly for CLEANSER.